19CSE362 · Fundamentals of Soft Computing · Team Presentation

Explainable ANN–ANFIS Hybrid for Renewable-Aware Task Offloading

Speaker script · 12 minutes · about 3 minutes each

🌍 Digital twin → 💾 Dataset → 📶 Network + baselines → 🧠 ANN → 🔗 ANFIS · 36 rules → ⭐ Hybrid + results

0:00	0:50	3:50	5:40	7:50	10:50
Rohith S	**Sanjith M P**	**Darshan R**	**Rohith S**	**Vishaanth R**	**Darshan R**
S1–S2 · Opening & problem	S3–S4 · Digital twin & dataset	S5–S6 · WiFi, edge & baselines	S7–S8 · ANN & ablation	S9–S11 · ANFIS, 36 rules, hybrid	S12 · Results & conclusion

🎯 How to use this script

1. Find your **coloured part**: it has your slides, your words and your handover line.
2. Read the **highlighted "Spoken script" box aloud** with a stopwatch; the time tags show where you should be.
3. Learn the **"If the reviewer asks you..."** table for your own part.
4. Everyone memorises the **Team appendix** numbers.

All numbers come from the project code · robustness & ablation: `robustness_ablation.py`

Course: 19CSE362 Fundamentals of Soft Computing · **Batch:** B.Tech ELC / CCE (2024) **Talk length:** 12 minutes, about **3 minutes per person**, following the data flow: **digital twin → dataset → network & baselines → ANN → fuzzy / ANFIS (36 rules) → hybrid → results**. Every number in this script comes from the project code. The 11-week robustness, Wilcoxon and ablation numbers come from `robustness_ablation.py` (run it again with `python robustness_ablation.py`, about 2 minutes).

📑 Contents

- 0 · Running order and slide plan
- PART 1 — Rohith S · Opening & the problem
- PART 2 — Sanjith M P · Digital twin & dataset
- PART 3 — Darshan R · WiFi, edge server & baselines
- PART 4 — Rohith S · ANN predictor & ablation
- PART 5 — Vishaanth R · ANFIS, 36 rules, hybrid & dashboard
- PART 6 — Darshan R · Results & conclusion
- PART 7 — Team appendix

0 · Running order and slide plan

Clock	Speaker	Time	Slides	Topic	Contribution form
0:00 – 0:50	**Rohith S**	0:50	S1–S2	Title, team, **the problem**: run locally or offload?	Opening (also wrote the paper)
0:50 – 3:50	**Sanjith M P**	3:00	S3–S4	Digital twin (solar, battery, ESP32, tasks), oracle cost, **dataset + 38 features**, 20/20 checks	Physics twin, dataset, validation
3:50 – 5:40	**Darshan R**	1:50	S5–S6	WiFi link + edge server models, the information gap, **Mamdani fuzzy** + **ANN-classifier** baselines	WiFi/edge, baselines
5:40 – 7:50	**Rohith S**	2:10	S7–S8	**ANN predictor** (inputs, 6-24-16-2, training, R^2), worked example, **ablation**	ANN predictor, ablation
7:50 – 10:50	**Vishaanth R**	3:00	S9–S11	**ANFIS** 5 layers, **36 rules**, hybrid learning, soft target, **hybrid** example, design ablation, **dashboard**	ANFIS layer, dashboard
10:50 – 12:00	**Darshan R**	1:10	S12	**Closed-loop results on 11 unseen weeks + Wilcoxon**, limitations, conclusion	Robustness, significance

Rohith speaks twice (0:50 opening + 2:10 ANN = 3:00) and **Darshan** speaks twice (1:50 + 1:10 = 3:00). This keeps the talk in data-flow order, and Darshan still presents his own robustness results at the end.

Slide list (12 slides)

#	Title	Owner
S1	Title, team, one-line idea	Rohith
S2	The problem: run locally or offload?	Rohith
S3	Digital twin: solar → battery → ESP32 → tasks	Sanjith
S4	Dataset generation, 38 features, oracle cost, 20/20 validation	Sanjith
S5	WiFi link and edge server models	Darshan
S6	The information gap + two baselines (Mamdani fuzzy, ANN classifier)	Darshan
S7	ANN predictor: inputs, architecture, training	Rohith
S8	ANN results, worked example, ablation	Rohith
S9	ANFIS: 5 layers, 36 rules	Vishaanth
S10	Hybrid learning, soft target, worked example	Vishaanth
S11	Design ablation + explainable dashboard	Vishaanth
S12	Results on 11 unseen weeks, significance, conclusion	Darshan

PART 1 OF 6

Rohith S · Opening & the problem

⏱️ **0:00 – 0:50 · Slides S1–S2 · about 110 words** *(Rohith returns at 5:40 for the ANN, Part 4)*

🔷 🖼️ What goes on your slides

S1 · Title

- *Explainable ANN–ANFIS Hybrid for Renewable-Aware Task Offloading in Solar-Powered IoT Nodes Using a Digital Twin*
- Team: Sanjith M P · Rohith S · Darshan R · Vishaanth R
- Chain graphic: ☀️ Solar PV → 🔋 Battery → 📟 ESP32 → 📶 WiFi/MQTT → 🖥️ Edge Server

S2 · The problem

- One IoT task arrives every minute (5 types, from sensor aggregation to image preprocessing)
- 💻 **LOCAL:** ESP32 CPU time + battery energy
- ⚡ **OFFLOAD:** radio energy + network delay + edge server delay
- The best choice changes with sun, battery, signal and server load
- 🎯 **Goal:** decide well using only what the node knows, *and explain why*

🔷 🗣️ Spoken script

`0:00` "Good morning, respected faculty. Our project is an *explainable ANN–ANFIS hybrid for renewable-aware task offloading in solar-powered IoT nodes, using a digital twin*. I am Rohith, and with me are Sanjith, Darshan and Vishaanth."

`0:15` "Picture a solar-powered ESP32 sensor node. Every minute a monitoring task arrives. It can run the task **locally**, spending CPU time and battery, or **offload** it over WiFi to an edge server, spending radio energy and waiting for the network and the server. The right choice keeps changing with the sun, the battery, the signal and the server load. Our goal is to choose well using only what the node knows, and to **explain why**."

🔷 🔁 Handover (0:45)

"Sanjith built a digital twin to study this at scale. Over to Sanjith."

◆ ❓ If the reviewer asks you (opening)…

Question	Crisp answer
What is task offloading?	Deciding *where* a computation runs: on the weak device itself, or on a stronger machine reached over the network. It trades CPU energy and time against radio energy and network/server delay.
Why is this a soft-computing problem?	The node must decide under uncertainty (it only knows the *last* signal and server load), and the trade-off is easiest to state in vague terms. ANN learning plus fuzzy reasoning handles both.
What is new in your work?	An ANN *predicts* the unknown offload cost, and an ANFIS with 36 learned rules *reasons* about it. The result is as accurate as a black-box ANN, uses less energy, and explains every decision.
Is there a cloud tier?	No: the two options are the ESP32 and an edge server. A cloud tier is future work.

PART 2 OF 6

Sanjith M P · Digital twin & dataset

⏱️ **0:50 – 3:50 · Slides S3–S4 · about 400 words**

🔷 🖼️ What goes on your slides

S3 · Digital twin (Δt = 1 min)

Model	Equation	Key values
Solar	G = 1000·sin(π(h−6)/12)·k_cloud ; P_PV = P_rated·G/G_STC·η	1 W panel, η = 0.85, OU clouds, Markov weather
Battery	SOC += η_c·P_ch·Δt/E_bat − P_dis·Δt/(η_d·E_bat)	12 Wh, η = 0.95, CC/CV taper > 90 %
ESP32	C = D·c ; T_local = C/(f(1−bg)) ; E_local = (P_idle + α·CPU%)·T_local	240 MHz, deadline 5 s
Tasks	5 types, 60 → 3000 cycles/byte ; size D follows a smooth AR(1) process, larger in working hours	1 task per minute
Closed loop	energy of the **chosen** option is drawn from the battery	

S4 · Dataset + oracle + validation

- Oracle cost: **J = λ(SOC)·E + T/5 s + 2·[miss]** (+ 0.3·PER + 0.3·U² for offload), with **λ = 1 + 4(1 − SOC)^1.5**
- 7 days × 1440 min = **10,080 tasks × 38 features** (seed 42, SOC starts at 25 %)
- Feature groups: Time & weather (4) · Solar & battery (8) · Task & ESP32 (9) · Network (10) · Edge (3) · Cost & decision (4)
- Every row stores **both** options in full (time, energy, cost) → ideal supervised-learning data
- Decisions emerge: **54 % LOCAL / 46 % OFFLOAD**
- Running example **T09206** (Anomaly Detection, 214 KB): local 1827 ms / 287 mJ vs offload 361 ms / 151 mJ → **OFFLOAD**
- `validate_twin.py` : **20 / 20 checks PASS**
- *Visual:* dashboard plot 7 (J_local vs J_offload scatter) or the SOC-vs-time plot

🔷 🗣️ Spoken script

`0:50` "Thank you, Rohith. Hardware tests at this scale are impractical, so I built a **physics-based digital twin** that advances one minute per step. For **solar**, irradiance follows a half-sine from 6 am to 6 pm, times a smoothly drifting cloud factor, with Markov-chain weather. The **battery** is a 12 watt-hour lithium-ion pack with 95 % efficiency and a CC/CV taper above 90 %.

There are **five task types**, from sensor aggregation at 60 cycles per byte up to image preprocessing at 3000, and task sizes follow a smooth AR(1) process that grows during working hours. On the **ESP32**, a task needs C equals D times c CPU cycles, that is, data size times cycles per byte. At 240 megahertz that gives the local time, and the local energy is idle power plus alpha times CPU load, multiplied by that time. Most importantly, the energy of whichever option is chosen is drawn from the battery, so every decision affects the next one. It is a **closed loop**."

`2:00` "The twin computes the true cost of both options. J equals a battery price lambda times energy, plus time over the 5-second deadline, plus a deadline penalty, and for offloading, link and server risk terms. Lambda equals 1 plus 4 times (1 minus SOC) to the power 1.5, so the same joule costs up to five times more on an empty battery. The cheaper option is the **oracle** decision. Running seven days gives our **dataset: 10,080 tasks with 38 features** in six groups: time and weather, solar and battery, task and ESP32, network, edge server, and cost and decision. Every row stores **both** options in full, which is exactly what supervised learning needs. About 54 % run locally and 46 % are offloaded, a split that emerges from the physics. Take task **T09206**, a 214-kilobyte anomaly-detection task: locally it would take 1.8 seconds and 287 millijoules, offloaded 361 milliseconds and 151 millijoules, so the oracle offloads it. We will follow this task through the whole talk."

`3:10` "To trust the twin, `validate_twin.py` runs **20 engineering checks**. For example, distance up means RSSI down, server load up means edge time up, the decision always equals the minimum cost, and the battery energy balance closes to within 10^{-5} %. All **20 pass**."

◆ 🔁 Handover (3:30)

"The offload side of the twin, the WiFi link and the edge server, was modelled by Darshan, who will explain it now."

◆ ❓ If the reviewer asks you…

Question	Crisp answer
Why a digital twin instead of a dataset from the internet?	We need the **true cost of both options** for every task, plus a closed loop where decisions change the battery. Public datasets record only one outcome per task.
Is the randomness realistic?	It enters only through physical processes: OU clouds, Markov weather, AR(1) task sizes, correlated shadowing, server bursts. Costs and decisions are never noisy. Lag-1 autocorrelation is 0.90 or higher for every signal.
Why λ(SOC)?	Energy is riskier when the battery is low. λ goes from 1 at a full battery up to 5 when empty, and it multiplies **both** options' energy.
Is the twin validated on hardware?	Not yet. It is verified for **internal consistency** (20 checks, a what-if table), and its parameters are datasheet-level. Hardware calibration is future work.

Question	Crisp answer
Which features does the AI actually use?	Only what the node knows **before** deciding: SOC, task size, complexity, local time and energy, **last** RSSI, **last** server load, hour. The true offload cost is used only as a training target.

PART 3 OF 6

Darshan R · WiFi, edge server & baselines

⏱️ **3:50 – 5:40 · Slides S5–S6 · about 260 words**

🔷 🖼️ What goes on your slides

S5 · Offload path models

Model	Equation	Key values
Distance	random walk 3–40 m (node on moving equipment)	mean 18 m
RSSI	−40 − 10·2.6·log10(d) + shadowing	σ = 4 dB, correlation 0.95
Rate	802.11n MCS7 65 Mbps @ −72 dBm … MCS0 6.5 Mbps @ −91 dBm	
Errors	PER = 1/(1 + 9e^{0.7·margin}) ; N_tx = 1/(1−PER) ≤ 7	10 % at sensitivity
Transmit	T_tx = bytes(+MQTT, TCP/IP)·8 / R_eff ; E_tx = 0.6 W·T_tx	
Edge	T_edge = C / (3 GHz·(1 − U))	explodes near saturation

S6 · The information gap + baselines

- The oracle needs the **current** RSSI, server load and true offload cost. The ESP32 knows only the **last** RSSI and **last** server load.
- 🌫️ **Mamdani fuzzy:** 4 inputs × 3 words (SOC, task load, WiFi link, server load) · trapezoid/triangle MFs · **12 expert rules** · AND = min · max aggregation · centroid
- 🧠 **ANN classifier:** 9 inputs → 24 → 16 → 1 sigmoid, cross-entropy on oracle labels (black box)
- Test protocol: each controller **drives the battery itself** on unseen weeks (closed loop)

🔷 🗣️ Spoken script

`3:50` "Thank you, Sanjith. I modelled the offload path. The node moves, so its **distance** to the access point wanders between 3 and 40 metres. **RSSI** follows log-distance path loss, minus 40 minus 26 log d, plus correlated shadowing. From the RSSI, 802.11n **rate adaptation** picks a speed from 65 down to 6.5 megabits per second. The **packet error rate** is 10 % at the sensitivity limit and falls with margin, which gives retransmissions. Transmission time is the bytes, with MQTT and TCP/IP headers, over the effective rate; radio energy is 0.6 watts times that time. At the **edge**, time equals cycles over 3 gigahertz times one minus the load, so it explodes as the server saturates."

`4:40` "Here is the key problem. The oracle needs the *current* signal and server load. A real ESP32 knows only the **last** measured RSSI and the **last** reported load. So I built two baselines that decide under this uncertainty. The first is a **Mamdani fuzzy controller** with four linguistic

inputs: battery, task load, WiFi link and server load. It uses trapezoid and triangle membership functions and **12 expert rules**, like 'IF the task is heavy AND the link is good, THEN offload', with min for AND, max aggregation and centroid defuzzification. The second is an **ANN classifier**: 9 inputs, two hidden layers, and a sigmoid output trained on the oracle labels."

5:25 "Every controller is tested in a **closed loop**, driving the battery through weeks it has never seen."

🔁 Handover (5:35)

"To decide well, the node must first know what offloading will cost. Rohith is back to show how we predict it."

❓ If the reviewer asks you…

Question	Crisp answer
Why log-distance path loss?	It is the standard indoor/industrial model. n = 2.6 fits cluttered spaces, and correlated (Gudmundson) shadowing makes obstacles appear gradually.
How did you choose the fuzzy membership functions?	Expert design on physical units. The RSSI breakpoints follow the MCS sensitivities (−86 / −80 / −74 dBm), and the task-load breakpoints are in ms relative to the 5 s deadline.
Mamdani or Sugeno for the baseline? Why?	Mamdani: its rules and output sets are in words, which is ideal for an expert, hand-written baseline. ANFIS (Vishaanth) uses Sugeno, because it is trainable.
Why is T_edge = C/(f(1−U))?	It is the M/M/1 sojourn-time form: service slows sharply as utilisation U approaches 1.
What happens if no fuzzy rule fires?	The score defaults to 0.5, which means LOCAL (OFFLOAD needs a score above 0.5).

PART 4 OF 6

Rohith S · ANN predictor & ablation

⏱️ **5:40 – 7:50 · Slides S7–S8 · about 305 words** *(your opening was Part 1)*

🔷 🖼️ What goes on your slides

S7 · ANN predictor ("predict" step)

- **Inputs (6):** ln(task size), ln(complexity), last RSSI, last server load, sin(2πh/24), cos(2πh/24)
- **Outputs (2):** ln(offload time [ms]), ln(offload energy [mJ])
- **Network:** 6 → 24 → 16 → 2 · tanh · linear output · **602 parameters** · plain NumPy
- **Training:** MSE · Adam (lr 0.01, β 0.9/0.999) · batch 128 · 60 epochs · inputs standardised
- **Split:** chronological 80 / 20 → 8,064 train / 2,016 validation (last 1.4 days, never seen)

S8 · Results + ablation

- Validation **R^2 = 0.989 (time), 0.987 (energy)**, median error **5.2 % / 5.6 %**
- Example T09206 (Anomaly Detection, 214 KB): predicted **353 ms / 149 mJ** vs real **361 ms / 151 mJ** (−2 %)
- Input ablation (validation R^2, time / energy):

Model	Time R^2	Energy R^2
All 6 inputs	**0.989**	**0.987**
− task size	0.924	0.906
− complexity	0.949	0.960
− last RSSI	0.968	0.957
− last server load	0.978	0.978
− hour	0.988	0.984

- System ablation (closed loop, unseen week): **without the ANN**, agreement 96.6 → **93.6 %**, deadline misses 9 → **14**, energy +25 J
- *Visual:* dashboard "ANN predictor" tab: predicted-vs-real scatter + learning curves

🔷 🗣️ Spoken script

5:40 "Thank you, Darshan. My part is the **ANN predictor**, the *predict* step. It estimates, *before* sending, the offload time and energy. The inputs are six things the node knows: the log of task size, the log of complexity, the last RSSI, the last server load, and the hour encoded as sine and cosine. The outputs are the log of offload time and the log of offload energy. We use

logs because these values span 10 milliseconds to 4 seconds, so the network learns **relative** error."

`6:15` "The architecture is **6–24–16–2**, with tanh hidden layers and linear outputs: 602 parameters, written from scratch in NumPy with back-propagation and the **Adam** optimiser, learning rate 0.01, batch 128, 60 epochs. The split is **chronological**: the first 5.6 days train, and the last 1.4 days are validation it has never seen."

`6:40` "The results: **R^2 of 0.989 for time and 0.987 for energy**, with a median error of about 5 %. Take Sanjith's task T09206, an anomaly-detection task of 214 kilobytes. The last RSSI was −68.8 while the actual signal was −62. Even so, the ANN predicts 353 milliseconds and 149 millijoules, against a truth of 361 and 151. That is within 2 %."

`7:05` "I ran an **ablation study**, removing one input at a time and retraining. Removing task size hurts most: time R^2 falls to 0.924. Complexity, last RSSI and server load follow, while the hour barely matters, at 0.988. Then at system level: if ANFIS is given the raw RSSI instead of my ANN's predicted cost, agreement with the oracle on an unseen week drops from **96.6 to 93.6 %**, and deadline misses rise from 9 to 14. The ANN is what turns uncertain network conditions into usable numbers."

🔁 Handover (7:40)

"But an ANN alone is a black box: it cannot tell us *why*. Vishaanth will show how ANFIS makes the decision explainable."

❓ If the reviewer asks you…

Question	Crisp answer
Why an ANN for this step?	The chain from last RSSI to MCS, PER, retransmissions, goodput and latency, plus the 1/(1−U) server effect, is strongly non-linear and step-wise. An MLP learns it from data without hand-written equations.
Why tanh, and why two hidden layers?	tanh is zero-centred and smooth, which suits standardised inputs. 24-16 is small enough for an MCU (602 parameters) yet reaches $R^2 \approx 0.99$.
How do you avoid overfitting / leakage?	A chronological split (no future data in training), validation monitored every epoch (train loss 0.0148, validation 0.0113), and inputs limited to decision-time values.
Why sin/cos for the hour?	The hour is cyclic: sin/cos keep 23:59 and 00:00 neighbours, which a raw 0–24 value would not.
Why not use the ANN classifier directly?	It is equally accurate (96.7 % vs 96.8 % over 11 weeks) but cannot explain itself, and it spends more energy (see Darshan's results).

PART 5 OF 6

Vishaanth R · ANFIS, 36 rules, hybrid & dashboard

⏱️ **7:50 – 10:50 · Slides S9–S11 · about 400 words**

🔶 🖼️ What goes on your slides

S9 · ANFIS: 5 layers, 36 rules

Input	Formula	Words
Battery SOC	SOC [%]	Low · Medium · High
Energy ratio	log10(Ê_offload / E_local) *(from ANN)*	Offload cheaper · Similar · Local cheaper
Time gain	100·(T_local − T̂_offload)/5000 *(from ANN)*	Small · Large
Server load	last server load [%]	Light · Busy

3 × 3 × 2 × 2 = 36 rules (one per combination)

Layer	Operation	Learned?
1 Fuzzify	$\mu = \exp(-(x - c)^2/2\sigma^2)$	✅ c, σ
2 Rule strength	w = product of μ (AND)	
3 Normalise	$\bar{w} = w/\Sigma w$	
4 Rule output	f = p·x + q (Takagi–Sugeno)	✅ p, q
5 Output	score = Σ w̄·f → OFFLOAD if > 0.5	

200 parameters = 20 premise + 180 consequent

S10 · Hybrid learning + example

- Each epoch: **least squares** for the 180 consequents → **gradient descent** for the 20 MF centres/widths · 30 epochs
- Soft target **t = 0.5 + 0.5·tanh((J_local − J_offload)/0.25)**: "how strongly" one option is better
- Trained on the ANN's **predictions**, so it learns to tolerate the ANN's errors
- T09206: inputs SOC 33.5 · energy −0.28 · gain 29 % · server 49 → top rule *SOC Medium · Offload cheaper · Small · Busy* (weight 0.19) → **score 0.9965 → OFFLOAD** (teacher 0.9945)

S11 · Design ablation + dashboard

Variant (unseen week)	Agreement	Energy	Misses
Full hybrid	**96.6 %**	932 J	9

Variant (unseen week)	Agreement	Energy	Misses
Hard 0/1 target	92.5 %	934 J	5
No MF tuning	97.0 %	935 J	9
No ANN	93.6 %	957 J	14

- Dashboard: 4 live controller cards · ANN prediction vs reality · **ANFIS rules that fired** · learned MFs · comparison charts
- *Visual:* screenshot of the "🔗 ANFIS" tab (learned membership functions + rule table)

🔶 🗣️ Spoken script

7:50 "Thank you, Rohith. My part is the *reason* step: **ANFIS**, the Adaptive Neuro-Fuzzy Inference System. ANFIS is a fuzzy rule base built as a five-layer network, so it can be **trained**. It has four inputs, and two of them come from Rohith's ANN. They are battery SOC, with the words Low, Medium and High; the **energy ratio**, the log of predicted offload energy over local energy, with Offload cheaper, Similar and Local cheaper; the **time gain**, Small or Large; and server load, Light or Busy. Three times three times two times two gives **36 rules**, one for every combination."

8:30 "Layer one uses **Gaussian membership functions**. Layer two multiplies them to get each rule's strength. Layer three normalises. In layer four each rule outputs a linear function, *p·x plus q*. That makes it first-order Takagi–Sugeno. Layer five takes the weighted sum, which is the **offload score**; above 0.5 means offload. In total there are 200 trainable parameters."

8:55 "Training uses Jang's **hybrid learning**. Each epoch, least squares solves the 180 rule-output parameters exactly, then gradient descent tunes the membership centres and widths, for 30 epochs. The target is a **soft oracle preference**, a tanh of the cost difference, so ANFIS learns *how strongly* one option is better. And because it is trained on the ANN's predictions, it learns to live with the ANN's errors. For task T09206, the energy ratio says offloading needs about half the energy. Its memberships are SOC Medium 0.73, Offload cheaper 0.51, Small gain 0.61 and Busy server 0.70; multiplied together, that rule's strength is 0.16. The strongest rule, *SOC Medium AND Offload cheaper AND Small gain AND Busy server*, carries 19 % of the weight, and the final score is **0.9965, so offload**. The teacher's value was 0.9945."

9:55 "We also checked my design choices. Training on hard 0-or-1 labels instead of the soft target drops agreement on an unseen week from **96.6 to 92.5 %**. Tuning the membership functions changes little: it is slightly better on validation and slightly worse on this test week. The least-squares rule outputs do most of the work, and we report that honestly."

10:20 "Finally, I built the **Streamlit dashboard**. It shows live cards for all four controllers, the ANN's prediction against reality, the **ANFIS rules that fired** with their weights, the learned membership functions, and the comparison charts. **Every decision is explained.**"

🔁 Handover (10:35)

"Darshan will now present how all the controllers performed."

❓ If the reviewer asks you…

Question	Crisp answer
Why ANFIS instead of a plain ANN?	The same accuracy, plus **explainable IF–THEN rules**. Trained on the soft target, it also makes cheaper mistakes: less energy than ANN-only in 11/11 weeks.
Why Gaussian MFs?	They are smooth and differentiable everywhere, so the gradients for the centres and widths exist.
Why 36 rules, and does that scale?	It is a grid partition: 3·3·2·2. More inputs grow the rules exponentially, so clustering-based rule generation (subtractive / FCM) is future work.
Explain hybrid learning precisely.	Forward pass: premises fixed, output linear in p, q → ridge least squares (exact). Backward pass: consequents fixed → MSE gradients on c and σ, with lr 30 and σ ≥ 0.15.
What do "Low/Medium/High SOC" mean here?	Learned centres at ≈ 18 / 29 / 41 %, because the training battery stayed between 15 and 43 %. Inputs outside that range are clipped.
Your ablation shows MF tuning barely helps. Why keep it?	Honest answer: its effect is within noise (validation 98.02 → 98.07 %, test week 97.0 → 96.6 %). It is kept because it makes the words fit the data, which keeps the rules readable. The real gains come from the ANN inputs (+3 points) and the soft target (+4 points).
The hard-target variant had fewer deadline misses (5 vs 9). Isn't it better?	Misses are small counts that vary a lot between weeks (std ≈ 7), so one week cannot separate them. Its agreement drops by 4 points, which is a large, systematic loss.

PART 6 OF 6

Darshan R · Results & conclusion

⏱️ **10:50 – 12:00 · Slide S12 · about 165 words**

🔷 🖼️ What goes on your slide

S12 · Closed loop on 11 unseen weeks (mean ± std)

Controller	Agreement with oracle	Task energy	Deadline misses	Mean regret
🎯 Oracle (upper bound)	100 %	991 J	4.9	0
🌫️ Fuzzy only	92.5 ± 0.8 %	1049 J	11.2	0.022
🧠 ANN only	96.7 ± 0.6 %	1024 J	9.5	0.012
⭐ **ANN + ANFIS**	**96.8 ± 0.4 %**	**1007 J**	**8.4**	**0.007**

Wilcoxon signed-rank (paired, 11 weeks)

- Hybrid vs Fuzzy: better on agreement, energy, regret in **11/11 weeks**, p ≈ 0.001
- Hybrid vs ANN only: energy (−17 J/week) and regret lower in **11/11 weeks**, p ≈ 0.001 · agreement **not significant** (p = 0.83)

Conclusion: as accurate as the best black box · cheaper decisions · explainable rules

🔷 🗣️ Spoken script

`10:50` "To evaluate fairly, every controller runs in a closed loop on **11 unseen weeks**, each with a new random seed, and we compare the paired results with the **Wilcoxon signed-rank test**. Mean agreement with the oracle is 92.5 % for fuzzy, 96.7 for the ANN alone and **96.8 for our hybrid**. Against fuzzy, the hybrid is better in all 11 weeks on agreement, energy and regret, with p about 0.001. Against the ANN alone, the accuracy difference is *not* significant. But the hybrid used **less task energy in all 11 weeks**, about 17 joules per week, and had lower regret, again with p about 0.001."

`11:35` "So our hybrid is as accurate as the best black box, makes cheaper mistakes, and **explains every decision**. The limitations: the twin is simulated, not yet calibrated on real hardware, and there is no cloud tier. Next we plan hardware calibration, a cloud tier, and running ANFIS on the ESP32 itself. Thank you; we are happy to take your questions."

❓ If the reviewer asks you…

Question	Crisp answer
Why Wilcoxon and not a t-test?	11 paired weeks is a small sample, and normality cannot be assumed. Wilcoxon is the non-parametric paired test. With n = 11, p ≈ 0.001 is the smallest value possible (every week favours one side).
Why closed loop instead of plain test accuracy?	In a closed loop a wrong decision drains the battery and changes all later decisions. That is the real deployment situation.
What is "regret"?	The average extra cost J of the chosen option compared with the best option. 0 means oracle-perfect.
Why do the battery curves look the same for all controllers?	Task energy is about 1 % of the node's weekly consumption (the 0.14 W always-on load is ≈ 85 kJ/week), so the controllers differ in task energy, misses and regret, not in SOC.
Can even the oracle miss deadlines?	Yes (4.9 per week on average). It minimises the weighted cost J, so when both options are slow, or the on-time one is very costly, a miss can be the cheapest choice.

TEAM APPENDIX

Team appendix

🔢 Numbers every member must know

Item	Value
Dataset	10,080 tasks × 38 features · 7 days · seed 42 · 54 % LOCAL / 46 % OFFLOAD
Twin validation	20 / 20 checks pass
ANN predictor	6-24-16-2 · R^2 0.989 / 0.987 · median error ≈ 5 %
Fuzzy baseline	4 inputs · 12 rules · Mamdani · centroid
ANN classifier	9-24-16-1 · sigmoid · cross-entropy
ANFIS	4 inputs · 36 rules · 200 parameters · hybrid learning · 30 epochs
Offline validation accuracy	Fuzzy 94.8 % · ANN 98.1 % · Hybrid 98.1 %
11 unseen weeks (agreement)	Fuzzy 92.5 % · ANN 96.7 % · **Hybrid 96.8 %**
Significance	Hybrid beats fuzzy on everything; beats ANN on energy & regret ($p \approx 0.001$); ties ANN on accuracy

⏱️ Rehearsal tips

- Speak at about 140 words per minute: each 3-minute part is about 420 words. Rehearse with a stopwatch, and cut the bracketed time marks if you run late.
- Use the **handover lines** exactly. They make the four parts sound like one talk.
- Use one example throughout, **task T09206**. Sanjith introduces it, Rohith predicts its cost, Vishaanth's ANFIS decides it.
- If you show the dashboard live, open `streamlit run streamlit_app.py` **before** the talk: the first load trains the models (a few seconds) and runs the comparison (about 10 seconds).
- Keep any live demo inside Vishaanth's slot (about 30 seconds on the 🔗 ANFIS tab).

⚠️ Make the contribution form match the evidence

- The form says Darshan ran "closed-loop tests over **11** unseen weeks" in one place and "**10-week** robustness" in another. Use **11** in both places (seeds 7, 11, 23, 99, 2024, 3, 5, 13, 17, 31, 101).
- The ablation and Wilcoxon numbers in this script come from `robustness_ablation.py` in the repository. If your IEEE paper has different numbers, present the paper's and keep them consistent.
- The IEEE manuscript and paper figures listed for Rohith are not in this repository. Bring them separately if the reviewers ask.